

Practical AES encryption in Go

The library

Next Steps: From Lab Code to Real Tool

AES-ECB

Step 1: The Imports

Step 2: Understanding Padding (PKCS7)

The `pkcs7Pad` Function

The `pkcs7Unpad` Function

Step 3: The `main` Function - Setup

Step 4: The Encryption Process (Manual ECB)

Step 5: The Decryption Process (Manual ECB)

AES-CBC (Common)

Step 1: The Imports

Step 2: The Padding Helpers

Step 3: The `main` Function - Setup

Step 4: The IV (Initialization Vector)

Step 5: Encryption with CBC Mode

Step 6: Decryption with CBC Mode

AES-CFB (Steam Cipher)

Step 1: The Imports

Step 2: The `main` Function - Setup

Step 3: The IV (Still 100% Necessary!)

Step 4: Encryption with CFB

Step 5: Decryption with CFB

AES-OFB

Step 1: The Imports

Step 2: The `main` Function - Setup

Step 3: The IV (Initialization Vector)

Step 4: Encryption with OFB

Step 6: Decryption with OFB

AES-CTR (Classic)

Step 1: The Imports

Step 2: The `main` Function - Setup

Step 3: The "Nonce" (IV)

Step 4: Encryption with CTR

Step 5: Decryption with CTR

AES-GCM (The Most Secure)

Step 1: The Imports

Step 2: Setup

Step 3: Create AEAD Cipher

Step 4: The Nonce (Critical!)

Step 5: Encryption ("Seal")

Step 6: Decryption ("Open")

The library

- `cipher.NewGCM` : A function from the `crypto/cipher` package. It takes a base cipher (like AES) and creates an AEAD (Authenticated Encryption with Associated Data) object. This is the modern, secure standard, as it provides both confidentiality (encryption) and authenticity (tamper-proofing).
- `cipher.NewCBCEncrypter` / `cipher.NewCBCDecrypter` : Functions that create a cipher object for Cipher Block Chaining (CBC) mode. This is a block cipher mode that "chains" blocks together to hide patterns. It requires an IV and PKCS7 padding.
- `cipher.NewCFBEncrypter` / `cipher.NewCFBDecrypter` : Functions that turn a block cipher into a stream cipher using Cipher Feedback (CFB) mode. It does not require padding and uses an IV.
- `cipher.NewOFB` : A function that turns a block cipher into a stream cipher using Output Feedback (OFB) mode. It also does not require padding and uses an IV.
- `cipher.NewCTR` : A function that creates a fast, parallelizable stream cipher using Counter (CTR) mode. It does not require padding and uses a Nonce.
- `aes.BlockSize` : A constant from the `crypto/aes` package. Its value is 16, representing the 16-byte block size that the AES algorithm operates on. This is why IVs are 16 bytes and why padding is needed for CBC.
- `crypto/rand` : A Go package that provides functions for generating cryptographically secure random numbers. It is used to create secure, unpredictable IVs and Nonces.

- `gcm.NonceSize()` : A method on a GCM object that returns the standard, recommended size for a GCM nonce, which is 12 bytes.
 - `iv` (Initialization Vector): A random block of data (usually 16 bytes for AES) used to initialize encryption modes like CBC, CFB, and OFB. It must be unique and random, but it does not need to be secret. Its purpose is to ensure that encrypting the same message twice produces two different ciphertexts.
 - `nonce` (Number used once): A value (like an IV) used to initialize a stream cipher like CTR or GCM. It MUST BE UNIQUE for every message encrypted with the same key. Re-using a nonce with the same key is a catastrophic security failure.
-

Next Steps: From Lab Code to Real Tool

- Practice all the code below and run all of them (Required)
 - **Make `mycrypto` Package:** Create a `mycrypto/utls.go` file. Copy *only* the GCM code into it to make `Encrypt` and `Decrypt` functions
 - **Build CLI Tool:** Use your new `mycrypto` package in `cli_tool/main.go` to make a tool that encrypts/decrypts files/message from the terminal. (Optional, reserved for future use)
-

AES-ECB

Step 1: The Imports

First, we need to import all the packages (libraries) our program will use.

```
package main

import (
    "bytes"      // For our padding helper
    "crypto/aes" // The core AES algorithm
```

```

"encoding/hex" // To make the ciphertext readable
"fmt"         // For printing to the console
"log"         // For handling fatal errors
)

```

Step 2: Understanding Padding (PKCS7)

AES is a **block cipher**, meaning it only encrypts data in fixed-size blocks (16 bytes for AES-128, -192, and -256). What if our message isn't a perfect multiple of 16 bytes? We must pad it.

We'll use a common standard called **PKCS7 padding**.

How it works: If you need to add `N` bytes of padding, you add `N` bytes, each with the value `N`.

- **Example:** If your block size is 8 and your data is `[10, 20, 30, 40, 50]`, you are missing 3 bytes. So, you add 3 bytes, each with the value `3`.
- **Result:** `[10, 20, 30, 40, 50, 3, 3, 3]`

The `pkcs7Pad` Function

This function adds padding to our data.

```

// pkcs7Pad appends padding to a byte slice to make it a multiple of the block
size
func pkcs7Pad(data []byte, blockSize int) []byte {
    // 1. Calculate how much padding is needed
    padding := blockSize - len(data)%blockSize

    // 2. Create the padding text itself
    // bytes.Repeat creates a new slice by repeating a byte N times
    padText := bytes.Repeat([]byte{byte(padding)}, padding)

    // 3. Append the padding to the original data
    return append(data, padText...)
}

```

The `pkcs7Unpad` Function

This function removes the padding to get our original message back.

```
// pkcs7Unpad removes PKCS7 padding from a byte slice
func pkcs7Unpad(data []byte) ([]byte, error) {
    // 1. Check for empty data
    if len(data) == 0 {
        return nil, fmt.Errorf("data is empty")
    }

    // 2. Get the last byte, which tells us how much padding there is
    padding := int(data[len(data)-1])

    // 3. Validate the padding
    if padding == 0 || padding > len(data) {
        return nil, fmt.Errorf("invalid padding")
    }

    // 4. Return the slice, cutting off the padding
    return data[:len(data)-padding], nil
}
```

Step 3: The `main` Function - Setup

Now we get to the main part of our program. Let's set up our key and plaintext.

```
func main() {
    // 1. Define our secret key.
    //   Must be 16, 24, or 32 bytes.
    //   This 32-byte key means we are using AES-256.
    key := []byte("example key 1234example key 1234")

    // 2. This is the message we want to encrypt.
    plaintext := []byte("This is a long message, more than one block!")
}
```

```

fmt.Println("Plaintext:", string(plaintext))

// 3. Create the AES cipher.
// This initializes the cipher with our key.
block, err := aes.NewCipher(key)
if err != nil {
    log.Fatal(err)
}

// Encryption and Decryption code will go here
}

```

Step 4: The Encryption Process (Manual ECB)

We are now ready to encrypt. We will do this block by block.

```

// 4a. Get the block size (it's 16 bytes for AES)
blockSize := block.BlockSize()

// 4b. Pad the plaintext so it's a multiple of 16
plainPadded := pkcs7Pad(plaintext, blockSize)

// 4c. Create a slice to hold the resulting ciphertext
ciphertext := make([]byte, len(plainPadded))

// 4d. Loop through the padded plaintext, one block at a time
for start := 0; start < len(plainPadded); start += blockSize {
    end := start + blockSize

    // Encrypt one block and store it in the ciphertext slice
    block.Encrypt(ciphertext[start:end], plainPadded[start:end])
}

// 4e. Print the (human-readable) hex version of the ciphertext
fmt.Println("Ciphertext (hex):", hex.EncodeToString(ciphertext))

```

Step 5: The Decryption Process (Manual ECB)

Decrypting is the exact reverse of encrypting.

```
// 5a. Create a slice to hold the decrypted (but still padded) data
decryptedPadded := make([]byte, len(ciphertext))

// 5b. Loop through the ciphertext, one block at a time
for start := 0; start < len(ciphertext); start += blockSize {
    end := start + blockSize

    // Decrypt one block and store it
    block.Decrypt(decryptedPadded[start:end], ciphertext[start:end])
}

// 5c. Remove the PKCS7 padding to get the original message
decrypted, err := pkcs7Unpad(decryptedPadded)
if err != nil {
    log.Fatal(err)
}

// 5d. Success!
fmt.Println("Decrypted:", string(decrypted))
}
```

AES-CBC (Common)

Step 1: The Imports

First, we need to import all the packages (libraries) our program will use.

```
"crypto/cipher" // NEW: Provides block cipher modes (like CBC)
"crypto/rand"   // NEW: For generating secure random numbers
```

Explanation:

- `crypto/cipher`: This package contains the implementations for different cipher modes. Instead of encrypting block-by-block manually, we'll tell this package to do it for us using the CBC algorithm.
- `crypto/rand`: This is a "cryptographically secure" random number generator. We'll use it to create a crucial piece of data called an **Initialization Vector (IV)**.

Step 2: The Padding Helpers

Please use the AES-ECB Padding (PKCS#7)

Step 3: The `main` Function - Setup

We start our `main` function just like before, by setting up our key, plaintext, and the core AES cipher block.

```
func main() {  
    key := []byte("example key 1234example key 1234") // 32 bytes  
    plaintext := []byte("CBC mode example with multiple AES blocks!!!")  
  
    fmt.Println("Plaintext:", string(plaintext))  
  
    // Create the AES cipher  
    block, err := aes.NewCipher(key)  
    if err != nil {  
        log.Fatal(err)  
    }  
    blockSize := block.BlockSize() // This will be 16 bytes  
  
    // ... Encryption and Decryption code will go here ...  
}
```

Step 4: The IV (Initialization Vector)

This is the most important new concept in CBC mode.

IV is A random, non-secret block of data. It MUST be the same size as the AES block (16 bytes).

```
// 1) Generate a random IV (Initialization Vector)
// The IV must be the same size as the block (16 bytes)
iv := make([]byte, blockSize)
if _, err := rand.Read(iv); err != nil {
    log.Fatal(err)
}
fmt.Println("IV (hex):", hex.EncodeToString(iv))
```

Step 5: Encryption with CBC Mode

Now, instead of looping manually, we use the `crypto/cipher` package.

```
// 2) Pad the plaintext and encrypt
plainPadded := pkcs7Pad(plaintext, blockSize)
ciphertext := make([]byte, len(plainPadded))

// Create a new CBC encrypter
// It takes the AES cipher and the IV
modeEnc := cipher.NewCBCEncrypter(block, iv)

// Encrypt all blocks at once
modeEnc.CryptBlocks(ciphertext, plainPadded)

fmt.Println("Ciphertext (hex):", hex.EncodeToString(ciphertext))
```

Look how much simpler that is! We just create a `NewCBCEncrypter` using our `block` and `iv`, and then call `CryptBlocks` once. The package handles all the chaining logic for us.

Step 6: Decryption with CBC Mode

Decryption is the exact reverse. We create a CBC *decrypter* using the **exact same key and IV**.

```
// 3) Decrypt
decryptedPadded := make([]byte, len(ciphertext))

// Create a new CBC decrypter using the same IV
modeDec := cipher.NewCBCDecrypter(block, iv)

// Decrypt all blocks
modeDec.CryptBlocks(decryptedPadded, ciphertext)

// Unpad the decrypted data to get the original
decrypted, err := pkcs7Unpad(decryptedPadded)
if err != nil {
    log.Fatal(err)
}

fmt.Println("Decrypted:", string(decrypted))
}
```

Now you get the AES-CBC

AES-CFB (Steam Cipher)

Step 1: The Imports

First, we need to import all the packages (libraries) our program will use.

Import the library the same way **as** above. ("**bytes**" is **NOT** needed!)

Step 2: The **main** Function - Setup

The setup is familiar: we define our key, plaintext, and initialize the AES cipher.

```

func main() {
    key := []byte("example key 1234example key 1234") // AES-256

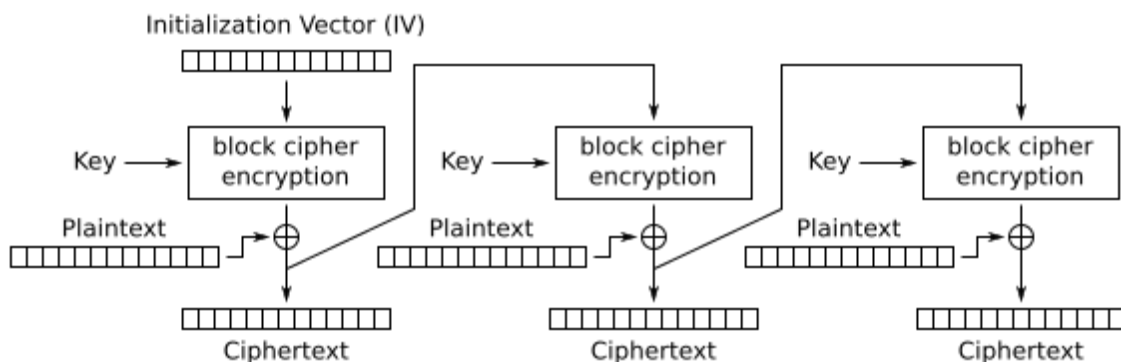
    // Notice our plaintext is not a multiple of 16!
    plaintext := []byte("CFB mode works like a stream, any length!")

    fmt.Println("Plaintext:", string(plaintext))

    block, err := aes.NewCipher(key)
    if err != nil {
        log.Fatal(err)
    }

    // ... Encryption and Decryption code will go here ...
}

```



Cipher Feedback (CFB) mode encryption

Step 3: The IV (Still 100% Necessary!)

CFB *must* have a random and unique **Initialization Vector (IV)** to be secure. If you re-use an IV with the same key, you create a massive security vulnerability

```

// 1) Random IV (same size as block)
// The IV must be 16 bytes for AES.
iv := make([]byte, aes.BlockSize)

```

```

if _, err := rand.Read(iv); err != nil {
    log.Fatal(err)
}
fmt.Println("IV (hex):", hex.EncodeToString(iv))

```

Step 4: Encryption with CFB

This is where things get different. Instead of `NewCBCEncrypter`, we call a `NewCFBEncrypter`. This function returns a `Stream` object. This object has a method called `XORKeyStream` which, as the name suggests, performs an XOR operation against our data.

```

// 2) Encrypt
// The ciphertext slice is the *exact same length* as the plaintext.
ciphertext := make([]byte, len(plaintext))

// Create a new CFB encrypter stream
streamEnc := cipher.NewCFBEncrypter(block, iv)

// Apply the keystream to the plaintext
streamEnc.XORKeyStream(ciphertext, plaintext)

fmt.Println("Ciphertext (hex):", hex.EncodeToString(ciphertext))

```

Step 5: Decryption with CFB

Decryption is the *exact same operation*.

```

// 3) Decrypt (use CFB Decrypter with same IV)
decrypted := make([]byte, len(ciphertext))

// Create a new CFB decrypter stream
streamDec := cipher.NewCFBDecrypter(block, iv)

// Apply the *same* keystream to the *ciphertext*

```

```
streamDec.XORKeyStream(decrypted, ciphertext)

fmt.Println("Decrypted:", string(decrypted))
}
```

AES-OFB

Step 1: The Imports

Once again, notice what's *not* here. Since OFB is a stream cipher, we do not need the `bytes` package for PKCS7 padding.

Step 2: The `main` Function - Setup

This setup is standard. We define our key and our plaintext. Note that the plaintext length is arbitrary it does not need to be a multiple of 16.

```
func main() {
    key := []byte("example key 1234example key 1234")
    plaintext := []byte("OFB mode example with any length message")

    fmt.Println("Plaintext:", string(plaintext))

    block, err := aes.NewCipher(key)
    if err != nil {
        log.Fatal(err)
    }

    // ... Encryption and Decryption code will go here ...
}
```

Step 3: The IV (Initialization Vector)

```
// 1) Generate a random IV (must be 16 bytes)
iv := make([]byte, aes.BlockSize)
if _, err := rand.Read(iv); err != nil {
    log.Fatal(err)
}
fmt.Println("IV (hex):", hex.EncodeToString(iv))
```

Step 4: Encryption with OFB

The code for this is beautifully simple. We create a `NewOFB` stream and call `XORKeyStream`. This one function handles generating the keystream and applying it to our plaintext.

```
// 2) Encrypt
ciphertext := make([]byte, len(plaintext))

// Create the OFB stream
streamEnc := cipher.NewOFB(block, iv)

// XOR the plaintext with the keystream
streamEnc.XORKeyStream(ciphertext, plaintext)

fmt.Println("Ciphertext (hex):", hex.EncodeToString(ciphertext))
```

Step 6: Decryption with OFB

Just like CFB, decryption is the *exact same operation*. We create a new OFB stream using the *same key and IV*. This generates the *exact same keystream*.

```
// 3) Decrypt (OFB uses same operation)
decrypted := make([]byte, len(ciphertext))

// Create a new OFB stream with the same IV
streamDec := cipher.NewOFB(block, iv)
```

```
// XOR the ciphertext with the keystream
streamDec.XORKeyStream(decrypted, ciphertext)

fmt.Println("Decrypted:", string(decrypted))
}
```

AES-CTR (Classic)

Step 1: The Imports

Once again, notice what's *not* here. Since OFB is a stream cipher, we do not need the `bytes` package for PKCS7 padding.

Step 2: The `main` Function - Setup

The setup is standard. We define our key and our plaintext. As always with a stream cipher, the plaintext can be any length.

```
func main() {
    key := []byte("example key 1234example key 1234")
    plaintext := []byte("CTR mode: counter-based, parallelizable, any length")

    fmt.Println("Plaintext:", string(plaintext))

    block, err := aes.NewCipher(key)
    if err != nil {
        log.Fatal(err)
    }

    // ... Encryption and Decryption code will go here ...
}
```

Step 3: The "Nonce" (IV)

In CTR mode or GCM, the "Initialization Vector" is typically called a **Nonce**, which stands for "**N**umber used **once**."

```
// 1) "Nonce" / initial counter: same size as block
// This is just like an IV and MUST be unique for each message.
nonce := make([]byte, aes.BlockSize)
if _, err := rand.Read(nonce); err != nil {
    log.Fatal(err)
}
fmt.Println("Nonce (hex):", hex.ToString(nonce))
```

Step 4: Encryption with CTR

just like our other stream ciphers.

```
// 2) Encrypt
ciphertext := make([]byte, len(plaintext))

// Create a new CTR stream
streamEnc := cipher.NewCTR(block, nonce)

// XOR the keystream with the plaintext
streamEnc.XORKeyStream(ciphertext, plaintext)

fmt.Println("Ciphertext (hex):", hex.ToString(ciphertext))
```

Step 5: Decryption with CTR

Just like OFB and CFB, decryption is the *exact same operation*.

We create a new CTR stream using the **same key and nonce**. This will generate the **exact same keystream** (`K1, K2, K3...`).

When we XOR this keystream with the ciphertext, we get our original plaintext back.

```
// 3) Decrypt: same operation again with same nonce
decrypted := make([]byte, len(ciphertext))

// Create a new CTR stream with the *same* nonce
streamDec := cipher.NewCTR(block, nonce)

// XOR the keystream with the ciphertext
streamDec.XORKeyStream(decrypted, ciphertext)

fmt.Println("Decrypted:", string(decrypted))
}
```

AES-GCM (The Most Secure)

We've learned about:

- **ECB**: Insecure (penguin problem).
- **CBC**: Secure block mode, but needs padding.
- **CTR/CFB/OFB**: Secure stream modes, no padding needed.

All of these modes had one critical weakness: they only provide **confidentiality** (privacy). They do **not** provide **authenticity**.

An attacker could (in many cases) flip bits in the ciphertext, and those changes would appear as garbage in the decrypted plaintext. They couldn't *control* the change, but they could *corrupt* the data, and you'd have no way of knowing until it was too late.

This is where **AEAD (Authenticated Encryption with Associated Data)** comes in.

Step 1: The Imports

First, we need to import all the packages (libraries) our program will use.

Import the library the same way as above. ("bytes" is NOT needed. it uses CT

R internally)

Step 2: Setup

We define our key and plaintext, then create the basic AES cipher block `aes.Cipher()`. This is the same as in all our other examples.

```
func main() {
    key := []byte("example key 1234example key 1234") // 32 bytes
    plaintext := []byte("GCM mode: authenticated encryption for real systems")

    fmt.Println("Plaintext:", string(plaintext))

    block, err := aes.NewCipher(key)
    if err != nil {
        log.Fatal(err)
    }
}
```

Step 3: Create AEAD Cipher

Instead of `NewCTR` or `NewCBC`, we create a `NewGCM`. This gives us a special `AEAD` object that handles both encryption and authentication for us.

```
gcm, err := cipher.NewGCM(block)
if err != nil {
    log.Fatal(err)
}
```

Step 4: The Nonce (Critical!)

Size: The GCM standard is 12 bytes.

We ask the `gcm` object for its preferred size (`gcm.NoncedSize()`), which will be 12.

```
// GCM nonce is usually 12 bytes
nonce := make([]byte, gcm.NoncedSize())
if _, err := rand.Read(nonce); err != nil {
```

```

    log.Fatal(err)
}
fmt.Println("Nonce (hex):", hex.EncodeToString(nonce))

```

Step 5: Encryption ("Seal")

This is the magic. The `gcm.Seal` function does everything in one step

```

// Encrypt: ciphertext includes auth tag at the end
// aad == nil for this simple example
ciphertext := gcm.Seal(nil, nonce, plaintext, nil)
fmt.Println("Ciphertext+Tag (hex):", hex.EncodeToString(ciphertext))

```

Step 6: Decryption ("Open")

The `gcm.Open` function is the reverse of `Seal`

```

// Decrypt: Verify tag and recover plaintext
decrypted, err := gcm.Open(nil, nonce, ciphertext, nil)
if err != nil {
    // This error is THE security feature!
    log.Fatal("Decryption failed (tag mismatch):", err)
}

fmt.Println("Decrypted:", string(decrypted))
}

```

GCM Full source code

```

package main

import (
    "crypto/aes"
    "crypto/cipher"

```

```

"crypto/rand"
"encoding/hex"
"fmt"
"log"
)

func main() {
    key := []byte("example key 1234example key 1234") // 32 bytes
    plaintext := []byte("GCM mode: authenticated encryption for real systems")

    fmt.Println("Plaintext:", string(plaintext))

    block, err := aes.NewCipher(key)
    if err != nil {
        log.Fatal(err)
    }

    // 1. Create the GCM AEAD cipher
    gcm, err := cipher.NewGCM(block)
    if err != nil {
        log.Fatal(err)
    }

    // 2. Create a unique 12-byte nonce
    nonce := make([]byte, gcm.NonceSize())
    if _, err := rand.Read(nonce); err != nil {
        log.Fatal(err)
    }
    fmt.Println("Nonce (hex):", hex.EncodeToString(nonce))

    // 3. Encrypt and "Seal" the message
    // The result is [ciphertext] + [auth_tag]
    // We pass 'nil' for "additional associated data" (AAD)
    ciphertext := gcm.Seal(nil, nonce, plaintext, nil)
    fmt.Println("Ciphertext+Tag (hex):", hex.EncodeToString(ciphertext))
}

```

```

// 4. Decrypt and "Open" the message
// We pass the same nonce and AAD.
// 'Open' will first verify the auth_tag.
decrypted, err := gcm.Open(nil, nonce, ciphertext, nil)
if err != nil {
    // If the tag is not valid (tampered message, wrong key, etc.)
    // 'Open' will return an error. THIS IS THE SECURITY!
    log.Fatal("Decryption failed (tag mismatch):", err)
}

// Only if err == nil is the data authentic
fmt.Println("Decrypted:", string(decrypted))
}

```